AUREA FORGE

Corintis
Engineering & Quality Diagnostic

PHASE 0

A readiness roadmap for tier-1 semiconductor, hyperscale and defense customers

Prepared for **Corintis SA** · Lausanne, Switzerland

By **Erick Polato** · Aurea Forge · epolato@aureaforge.com

17 August 2026

V1.0 · FINAL

CONTENTS

How to read this document

> **One evidence base, three lenses, in reading order.** Section (b) is the primary analysis: **engineering practice** across four dimensions, with the evidence behind every statement. Sections (b2) and (b3) then re-read that same evidence through **ISO/IEC 25010** (product quality) and **ISO 9001** (process quality), so the findings connect to the quality-management vocabulary already in use at Corintis. The two standards views deliberately sit *after* the analysis: each of their ratings is a summary of evidence presented in (b), so they read as conclusions rather than assertions.
>
> Section (a) is the standalone executive summary for readers who need the decision rather than the derivation. Every observation is traced to concrete evidence and paired with a recommended action, an effort estimate and a priority. The document is written to be used as a working plan, not filed as an assessment.

SECTION A

Executive summary

Corintis enters this diagnostic from a position of **real engineering strength**. It has built a **world-class simulation solver** on solid software-engineering foundations: gold-standard code verification with Method of Manufactured Solutions and convergence-order assertions, a development flow where 100% of recent solver work went through pull request and 80% was peer-reviewed, and a strict, well-configured quality gate on the product frontend that passes all eight of its conditions. This is not an organization that needs fixing.

It is an organization at an **inflection point**. The engineering practices that carried Corintis from EPFL spin-off to a commercial product are not yet the practices that tier-1 semiconductor, hyperscale and defense buyers audit before they sign. **That distance is the opportunity this report maps**, and the good news is that it is narrower than it looks: most of it is structural rather than cultural, and several of the highest-value moves cost a line of configuration rather than a quarter of engineering.

The opportunity concentrates in one place: a **structural asymmetry** between the scientific core and the product surface (Glacierware cloud and desktop). That asymmetry has a single measurable root cause, and naming it precisely is what makes it addressable:

> **The commercial cloud platform has a bus factor of 1.** A single engineer is the author of ~75% of the product repositories: the entire backend, the config server, the production-secrets repository, and every worker service. The solver's code, by contrast, is spread across **27 distinct contributors** (current team of 11, no single author above 33%).
>
> **This is not a discipline gap, it is a capacity gap**, and capacity is something a company can decide to invest in.

Most of the other observations in this report trace back to that single cause: peer review and effective quality gates are largely absent on the cloud side, on the backend because there is no second engineer to review, release traceability has not been established, and one credential reached production in plaintext. Read together, they are not a list of failures. They are what a platform looks like when one person carries it further than any one person should be asked to.

Three takeaways for leadership

1. **People unlock process, so sequence them in that order.** Peer review and enforced quality gates become available on the cloud platform the moment a second engineer exists, and not before. Cloud software engineering capacity plus a DevSecOps profile is therefore lever #1: it is what converts every process recommendation in this report from aspiration into something the team can actually operate. Framed for the board, this is **business continuity** on the commercial platform.

2. **The solver's validation is the largest untapped commercial lever.** Numerical V&V is excellent: Method of Manufactured Solutions, convergence-order assertions, and Taylor tests that verify the adjoint gradients. What is missing is **structured physical validation**: today it happens ad hoc (the solutions team compares flow rate and pressure/temperature deltas against Fidelity, Celsius and OpenFOAM, plus some lab data, scattered across Notion), with **no uncertainty quantification**. Because the physics and geometries are novel (microchannel cooling), little external reference data exists, which makes a rigorous **ASME V&V 20** program *more* necessary, not less. This is the largest commercial-value opportunity in the diagnostic.
3. **"Green CI is not a quality gate."** No repository blocks a *merge* on tests or SonarCloud, and 8 of 16 repositories have no branch protection at all. The clearest illustration comes from the *best-configured* repository: the frontend has a genuinely strict SonarCloud quality gate (8 conditions, coverage floor 80%, zero critical, zero major) that **currently cannot fail the CI job**, because `sonar.qualitygate.wait` is not enabled. The gate exists as a dashboard, not as a control. Turning it into a real gate costs **one line of configuration**.

Operating constraint: the hiring freeze

A company-wide **hiring freeze** is in effect (both the approved Product Owner role and the DevSecOps role are on hold; confirmed 2026-07-06). The freeze does not change the root cause, it makes it more urgent, and it forces a **two-track approach**: de-risk now without new headcount (cross-train a second engineer onto the backend, runbooks and documentation, low-cost process fixes) while keeping the **hiring business case ready** for the moment the freeze lifts.

> **Developments since the interviews (as of 2026-08-05).** Two changes improve the outlook and are reflected in the roadmap. First, investor conversations have been positive and **positions are expected to open sooner than the original freeze assumption**, which moves the hiring business case from contingency to near-term preparation. Second, an engineer from the **Solutions team has been assigned to act as Product Owner at 50%**. That is a well-judged interim move: the intake problem described in Dimension 2 is largely caused by demand arriving through Solutions, so placing the role with someone who already holds the customer conversations addresses the root of it rather than adding a layer on top. The recommendations below treat this as a bridge, not a substitute: a half-time Product Owner can establish a single intake and a triaged backlog, and should be resourced with an explicit decision about which half of their Solutions workload is set down.

This report also anchors its recommendations to the business north star: **cold-plate performance** (Glacierware is valued as an enabler of better cold plates). Every quality investment recommended here *protects and accelerates cold-plate leadership*; none is proposed as abstract quality.

Intellectual-property risk (advisory; legal track in motion)

The exposure is not only the decompilable Python-packaged binary: **Firedrake's LGPL license** may also affect proprietary algorithms. Legal counsel is already involved. This is a combined legal and packaging topic; it is flagged here as advisory and is not built into the Phase 0 scope.

> **IMMEDIATE ACTION · P1**
> Rotate the Zendesk API token committed in plaintext in `glacierware-prod.properties`, purge it from git history, and enable organization-wide **secret scanning with push protection**. Worth noting that every *other* secret in that repository is correctly encrypted, so the right pattern is already established and this is a single exception to close rather than a practice to build. Secret hygiene is table stakes in tier-1 supplier security reviews, which makes this both the fastest and the highest-signal win available. Three separate actions have to be confirmed, because they are not the same thing: the token **rotated or revoked**, the git history **purged**, and **secret scanning with push protection enabled** org-wide. Raised with the Head of Software Engineering on 2026-08-05 and assigned; **still open at the date of issue**, with remediation expected the following week. This is the only finding in this report that describes a live defect.

Maturity scorecard (1 to 5)

Ratings use a single scale throughout this report, applied identically to the four dimensions and to the ISO/IEC 25010 characteristics in section (b2). The distinction that matters most for Corintis is between level 3 and level 4: **a practice that is defined is not the same as a practice that is enforced**, and much of what follows sits precisely on that boundary.

Level	Definition	What it looks like in evidence
1	**Absent or ad hoc**	No defined practice; outcomes depend entirely on individual initiative
2	**Emerging**	The practice exists in some places, applied inconsistently, not defined organization-wide
3	**Defined**	Documented and agreed, but not enforced by tooling and not measured
4	**Enforced and measured**	Mandatory, automated where possible, and tracked over time
5	**Optimizing**	Measured, reviewed by leadership on a cadence, and continuously improved

#	Dimension	Maturity	Headline
1	Quality strategy & gates *(in depth)*	**2.5**	Solver ~4.5 / cloud ~1.5; gates do not block
2	Operating model & ways of working	**2.0**	Disciplined solver / "direct-to-main" cloud
3	Metrics & indicators	**2.0**	Framework designed but not operated; no KPI owners assigned; DORA not instrumented
4	Organizational design & roles	**2.0**	Bus factor 1 on the cloud (solver ~4 / cloud ~1)

The variance matters far more than the average. These numbers are not a verdict on the team, they are a map of where investment converts fastest into commercial capability. Read correctly, the scorecard says: **there is a strong technical core and a product surface that has outgrown the resources assigned to it. Close that distance with a structure that scales.**

The commercial lens: what tier-1 customers audit

Corintis is selling into a market defined by chipmakers, hyperscalers and defense primes. Buyers of that size do not simply evaluate a product, they **qualify a supplier**: technical due diligence, supplier-quality audits, and security questionnaires that ask an engineering organization to evidence how it works, not just what it built. That is the standard this roadmap is calibrated against, and it is the most useful way to read every finding that follows.

What a tier-1 customer asks	Where Corintis stands today	Addressed in
"Show me your simulation is validated against physical reality, with quantified uncertainty."	Numerical verification is gold-standard. Physical validation is real but informal, and uncertainty is not yet quantified.	D1 · #6
"Show me the gate a release has to pass."	The solver has a strong, auditable, multi-stage gate. On the cloud, gates are well defined but not yet wired to enforce.	D1, D2 · #1b, #5
"What happens to our program if a key engineer leaves?"	The commercial platform currently rests on one engineer.	D4 · #3, #3b
"How do you manage secrets, and how is our IP protected in what you ship?"	One credential reached production in plaintext (remediation underway); organization-wide secret scanning is not yet enabled; packaging and licensing sit on the legal track.	D1 · #1, #11
"Show me your engineering and quality indicators for the last two quarters."	The tooling is in place and well configured. A leadership-owned indicator framework is not yet established.	D3 · #8
"Is your quality system documented, reviewed and internally audited?"	ISO/IEC 25010 is adopted as the product-quality model, and the **software-scope internal-audit self-assessment has been drafted** and is pending review with the quality organization. What remains beyond that is the measurement layer and a recurring management review.	a3 · #8, #9

None of those answers is a "no". Every one is a *"partly, and here is the plan"*, which is exactly the position from which a supplier qualification is won rather than lost. **The work in this report is what turns a strong engineering team into an auditable supplier** at the scale Corintis is targeting for 2026.

SECTION B

Analysis by dimension

Dimension 1 · Quality strategy & quality gates *(in depth)* · Maturity 2.5/5

Current state. Two realities. **Glaciercore**: 420 test files, including a *verification* suite built on the Method of Manufactured Solutions (thermal and flow, 2D and 3D) with convergence-order assertions (>1.9): gold-standard code verification. **Cloud**: ranges from 0 tests (several billing/compute services) to reasonable suites (backend 99 tests, frontend ~327), but with gates that do not block.

Where to strengthen.

- **Solver V&V (ASME V&V 20):** code verification mature ✅ ; **solution verification** partial; topology-optimization **gradient checks do exist** ✅ (Taylor tests of the adjoint); **physical validation ad hoc** ⚠️ (owned by the solutions team, scattered across Notion; we reviewed one such artifact, the SW-vs-lab hydrodynamic validation: real experimental work with measurement uncertainty, but informal, exploratory, and with no acceptance criterion); **uncertainty quantification absent** ❌ . In addition, the mathematics is coupled to the solver stack, so math errors surface late, as divergence or as *silently wrong physics*. Verification runs only on push to `develop` ; it is not a gate.
- **"Green CI is not a gate":** no repository requires tests to merge; JaCoCo is configured without thresholds; `config-server` and `json.builder` build with `-DskipTests` .
- **The SonarCloud quality gate is strict, well configured, and not wired to block.** Credit first: the organization gate carries **eight demanding conditions** (coverage ≥ 80%, zero critical, zero major, all ratings ≤ A, duplication ≤ 3%, 100% of security hotspots reviewed) and the frontend **meets every one of them comfortably**: **86.1% coverage**, zero bugs, zero vulnerabilities, five minor code smells and **42 minutes** of estimated remediation debt across 96,844 lines of code. That is solid engineering by any standard. The gap is purely one of wiring: neither the frontend (`sonarqube-scan-action`) nor the backend (`sonar-maven-plugin`) sets `sonar.qualitygate.wait` , so the Sonar job turns green as soon as the analysis uploads, *without waiting for the verdict*. The frontend's `deploy: needs: sonarcloud` therefore gates the release on "tests ran and analysis uploaded", **not** on "the gate passed": **a red quality gate would still deploy**. The health report itself evidences this, its analysis-events field reads *"Green (was Red)"*. What *does* protect the frontend today is that a failing `pnpm run test:coverage` fails the job and blocks the deploy, making it the one cloud repository with a genuinely test-backed release gate, and therefore the **internal template** for the rest.
- `config-server.cloud` **is the highest-risk release path:** no tests in CI, yet it decrypts and serves configuration and secrets to every service at boot, so one broken commit can **take the whole platform down**. It also has **no branch protection**. (Nuance confirmed by the platform owner: Heroku waits for green CI before deploying, but config-server CI is compile-only `-DskipTests` , so a config that compiles yet is wrong still reaches production.) The operations review (corrected version, 2026-08) confirms the blast radius: if the config server is down, Spring Boot services fail to start or start without usable configuration, and a wrong configuration **fails silently**.
- **P1 security:** a Zendesk API token sits in plaintext in dev and prod property files (line 17) and in git history; every other secret in that repository is correctly `{cipher}` -encrypted, which shows the right pattern exists and this is a fixable exception.

Recommendations. (P1) Rotate the secret, enable secret scanning, fix config-server CI. (P1/P2) Make tests a required status check on protected branches. (P2) Structure the ad hoc validation into a **V&V 20 program** (defined error metrics, uncertainty quantification, one documentation home) and make verification a gate; establish a test baseline for billing/compute services.

Dimension 2 · Operating model & ways of working · Maturity 2.0/5

Current state (quantified via the GitHub API). The solver flow is mature: **100%** of recent work via PR, **80%** reviewed, 85% Conventional Commits, 481 issues under management. The cloud, in contrast: most services push **directly to** `main` ; **5 repositories have never merged a single PR**; the core API repository shows **0% review and 100% instant self-merge**. Self-merge averages ~85% across the cloud.

Where to strengthen. No organization-wide merge policy; **0 of 16** repositories require tests; **8 of 16** have no branch protection; no shared PR template (only the solver has one); release tagging abandoned outside the solver and the frontend (the backend's last tag is 2025-03 with 578 commits since; the worker services carry a perpetual `0.0.1-SNAPSHOT` and no tags). The declared protection of the frontend (1 review) does not match observed behavior (10% reviewed).

Confirmed in interview (2026-07-06): the operating model is **solver-first** (features are prototyped in Glaciercore and ported to Glacierware once proven), which makes the cloud a follower and adds to its load. There is **no Definition of Ready or Done**; each team works its own way.

There are **no sprints** (tried early 2026, did not stick). Glaciercore plans in GitHub Projects while Glacierware uses a Notion Kanban: two tools, no unified flow visibility. The solver team runs product development, research and technical support through one undifferentiated board, with no work-in-progress limits.

> **An important clarification on this point, following discussion with the Head of Software Engineering.** The team deliberately values everyone being able to work on everything, and that is a genuine strength worth protecting: cross-skilled engineers are exactly what keeps key-person dependency low, which is the opposite of the problem identified on the cloud side. **The recommendation here is not to split the team, nor to assign people to fixed lanes.** It is to **classify the work**: product development, research and customer support are three different types of demand, with different urgency, different predictability and different business value. When they share one queue with no labels, three things become impossible: protecting a known share of capacity for product work, telling leadership why a roadmap item slipped, and measuring flow for any of the three. All of that can be fixed *on the same board*, with the same people, using swimlanes or labels per demand type plus an agreed capacity allocation. The flexibility stays; only the visibility changes. The team already holds the right instinct here, having identified isolating product development and reinstating sprints as its own next step; this recommendation validates and structures that instinct.

Feature-request flow (current state, from the Head of Software Engineering's own diagram): intake is uncontrolled, in his own words *"a bit all over the place"*: multiple entry points (customers via the solutions team, the Therminator tool, and the CEO injecting ideas directly), person-to-person routing with no triaged backlog, and the solutions team self-implementing scripts (fast, but low-visibility work of which only some is later folded into the products: shadow work and technical debt). This is the clearest operating-model opportunity in the diagnostic and the direct justification for a **Product Owner** owning a single intake, triage and prioritization: the same role that turns scattered demand into a plan for how the software generates revenue.

Recommendations. A minimum merge policy (PR + 1 review + green tests required). **This splits in two:** on `app.frontend.corintis` it is available *today*, because that repository has two real contributors, so the gap there is between declared protection (1 review) and observed behaviour (10% reviewed) and can be closed this week; on the backend it genuinely waits for a second engineer, since review cannot be required of a sole author; org-wide PR template and CODEOWNERS; a branching and release policy; branch hygiene in Glaciercore (589 remote branches).

Dimension 3 · Metrics & indicators · Maturity 2.0/5

Current state. Tools exist (SonarCloud, Codecov, JaCoCo) but the indicator framework is **defined but not in force**. A complete GQM tree with operating rules exists, and two dashboards have been built with documented counting rules. **Revised upward from an earlier reading of 1.5**, which described the framework as absent; the evidence shows it is designed and not operated. The real gap is narrower and more fixable: **none of the eight KPI owners is assigned and the weekly review cadence is not running**, and DORA itself is still not instrumented. A defined metric that nobody executes does not evidence conformity, but it is a far shorter distance to close than building the framework from nothing.

Where to strengthen. Two of the four DORA keys are **not measurable** (change-failure rate and MTTR: there is no incident log); the other two exist only as proxies (lead time ≈ time-to-merge: solver 26.5h, frontend 21.4h, backend ~0h through self-merge; deployment frequency has no record). Release tagging is abandoned outside Glaciercore and the frontend.

> **A clarification worth stating explicitly, because it is the first question this section raises.** The four DORA measures **do not measure technical debt.** They measure delivery throughput and stability, and they reveal debt only indirectly and late, as lengthening lead times and a rising change-failure rate. Technical debt is a product-quality property and belongs under ISO/IEC 25010 *Maintainability* in section (b2). It also separates into three kinds that need different instruments: **rule-level debt**, which SonarCloud already measures and reports as 42 minutes of remediation effort on the frontend; **structural debt**, which no tool measures automatically and which is where the real exposure sits, in the untested services, the config-server single point of failure and the mathematics coupled to the solver stack; and **process debt**, the absent release traceability and incident record. The 42-minute figure is accurate and reassuring about the first kind only, which is why Maintainability is rated 2 rather than 4.

Confirmed (SonarCloud technical-health report, 2026-08-04): the frontend stands at **86.1% coverage** (analysis of 2026-07-31) against an **80% gate floor that exists but does not block**. This revises *upward* the provisional ~66% figure that had been circulating (a local measurement flagged as temporary). The exact Sonar thresholds, previously an open item, are now **documented and closed**: eight demanding conditions, all green. Backend coverage remains unknown.

Confirmed (operations review, corrected 2026-08): incident detection and recovery are **manual, with no automated rollback** (rollback = redeploy the previous Heroku image), which confirms the MTTR gap.

Confirmed (leadership): the north star is **cold-plate performance**; Glacierware is valued as an enabler. Today **no engineering-health metric exists**, only the product outcome. The GQM tree should therefore anchor every KPI to "cold-plate performance leadership" (where solver V&V serves the business directly) and to "Glacierware enables design velocity".

Recommendations. Stand up a **minimum dashboard from data already available** (review rate, share of gated repos, lead time, coverage) with a leadership owner; a **GQM tree** tying each KPI to the 2026 goals; instrument deploy events (deployment frequency) and an incident log (change-failure rate, MTTR).

Dimension 4 · Organizational design & roles · Maturity 2.0/5

Current state (17 people, confirmed org chart). The solver (**Glaciercore, 11**) is a healthy stream-aligned team with real software and computational depth (2 senior software engineers plus several computational engineers/scientists and interns). The product side (**Glacierware, 6**) is design-led: **3 of the 6 are designers**; the engineers are the Product Engineering Lead, a full-stack engineer and a front-end engineer, and **only one of the three is backend-capable**. The entire Spring Boot backend, config server and worker fleet therefore sits inside a design-led team and rests on one engineer: a breadth of ownership beyond what any single role can carry indefinitely, and a **single point of dependency across the commercial surface**. Two independent sources (git authorship and the org chart) converge on this, which is what makes it actionable rather than anecdotal. Structurally, the **platform function does not yet have a home of its own**: there is no platform team and no enabling quality function, so backend and infrastructure work is absorbed by a team organized around the product interface. (This quantifies the asymmetry: of 17 people, ~5 are software engineers, and backend capacity for the commercial cloud is effectively one person.)

> **How to read this finding.** This is a **structural bus-factor risk, not an individual performance finding**. One engineer single-handedly built the entire commercial cloud platform, which is a remarkable achievement. The risk is organizational: the company let a commercial platform rest on one person without support or gates. It should be read, and resourced, as **business continuity**.

Where to strengthen. Key roles unfilled (**Product Owner and DevSecOps on hold due to the hiring freeze**; QA planned); ownership model undefined (CODEOWNERS on only a few repositories).

Recommendations (roles). +1 to 2 cloud software engineers (the #1 structural fix; unlocks review at all); **DevSecOps** (the highest-leverage specialist: closes the config-server SPOF, secrets, environment separation and release gates in one role); **Product Owner** (single intake, refinement, DoR/DoD); **QA/QE** leveraged with AI tooling (test strategy plus the KPI dashboard); the Director of Global Quality accountable for the KPI framework. Full RACI in section (d).

SECTION B2

Product-quality view · ISO/IEC 25010:2023

Section (b) above gives the *engineering-practice* view. This section re-reads exactly the same evidence through the nine product-quality characteristics of **ISO/IEC 25010:2023**, the framework adopted by Corintis's quality leadership, so that the findings can be discussed in product-quality terms.

How to read a rating. The scale is the one defined in section (a): 1 absent, 2 emerging, 3 defined but not enforced, 4 enforced and measured, 5 optimizing. Each row below states *what holds the rating down* and points to the dimension in section (b) where the supporting evidence sits, so a rating can always be traced rather than taken on trust. Two methodological notes: ISO 25010 models the *product*, so these ratings are inferred from practice evidence plus direct observation; and the three characteristics marked with an asterisk fall outside the agreed deep scope of this diagnostic, so they are recorded as **provisional** and flagged for a light follow-up assessment rather than presented as measured results.

#	Characteristic	Rating	Basis for the rating, and where the evidence sits
1	Functional Suitability	**3**	Features work and the solver's *numerical* verification is strong (MMS, convergence order >1.9). Held at 3 rather than 4 because **physical validation is ad hoc and carries no acceptance criterion**, so correctness on novel geometries is demonstrated by expert judgement rather than by evidence a customer could audit. *See D1.*
2	Performance Efficiency	**~4***	Cold-plate performance is the company's core technical strength and the solver is HPC-tuned. Recorded as ~4 on observation. ***Provisional: not formally assessed in this engagement**, and worth confirming so it can be evidenced in a customer audit.
3	Compatibility	**3**	Services interoperate reliably through shared MongoDB and the config server, with no interoperability incidents reported. Held at 3 because the integration contracts between services are **not covered by integration tests**, so compatibility is observed in production rather than verified before release. *See D1.*
4	Interaction Capability *(usability)*	**~4***	Half of the product team are designers, which makes usability a likely strength. ***Provisional: outside the deep scope** of this diagnostic.
5	**Reliability**	**2**	🔴 Emerging rather than defined: the config server is a **single point of failure whose CI skips tests**, incident recovery is manual with **no automated rollback**, and several services including billing and compute-job submission carry **no tests at all**. Nothing here is undefined by accident, it simply has not been built yet. *See D1, D3.*
6	**Security**	**2**	🔴 The encryption pattern is right (`{cipher}` across the config store) but **one credential reached production in plaintext** and **no organization-wide secret scanning** exists to catch the next one, so the control depends on individual diligence rather than on tooling. The packaging and LGPL licensing questions remain open on the legal track. *See D1.*
7	**Maintainability**	**2**	🔴 Driven by structure rather than by code quality: **one engineer authors ~12 of 16 repositories**, most services have neither tests nor runbooks, and the solver's mathematics is coupled to its stack, which limits testability. *Important counter-evidence: the frontend is Sonar-rated A with 0.3% duplication and 42 minutes of remediation debt, so the capability plainly exists wherever it is measured and resourced. See D1, D4.*
8	Flexibility	**3**	dev and prod are properly separated (separate Heroku apps and databases) and the desktop app is cross-platform via Tauri. Held at 3 because **scaling is limited by team capacity** rather than by architecture, and there is no staging tier between dev and prod. *See D1, D4.*
9	**Safety** *(new in 2023)*	**2.5**	🟠 The specific exposure is **"silently wrong physics"**: because the mathematics is coupled to the solver stack, an error can surface late as a plausible-looking result rather than as a failure. Combined with unstructured validation, that is a **trust risk with defense and semiconductor customers**, whose own engineers will interrogate the tool. Rated above Reliability and Security because the underlying numerical verification is genuinely strong. *See D1.*

Where to focus: the highest return sits in **Reliability, Security and Maintainability**, all three on the cloud/practice side and all three addressed by the same underlying investment. **Functional Suitability and Safety** are the solver-side characteristics that matter most to defense and semiconductor buyers, and they are governed by the V&V program in Dimension 1. The three probable strengths (Performance, Interaction Capability, Compatibility) are worth confirming with a light assessment so they can be *evidenced* rather than asserted in a customer audit.

SECTION B3

Process-quality view · ISO 9001 clauses

ISO 9001 addresses quality as a *management system* and so complements the product view of ISO 25010. This is a conceptual mapping of the process findings onto the standard's clause structure, intended to make the diagnostic usable inside Corintis's existing quality system. It is not a full 9001 audit.

> **Why the numbering starts at 4, not at 1.** ISO 9001 follows the harmonized Annex SL structure, in which **clauses 1, 2 and 3 are introductory**: Scope, Normative references, and Terms and definitions. They describe what the standard covers and define its vocabulary, and they contain **no auditable requirements**. The requirements an organization is actually assessed against begin at **clause 4 and run to clause 10**, which is why any assessment, including an internal audit, starts there. Where a requirement sits at sub-clause level, the sub-clause number is used (for example 7.1.6, Organizational knowledge) so the reference can be looked up directly in the standard.

ISO 9001 clause	State at Corintis (software)	Opportunity
4 · Context of the organization	The commercial context is well understood, but the software organization has **no documented QMS scope and no process map**, and the audit requirements of tier-1 customers are not yet translated into documented quality requirements	Define the software QMS scope and a process map. The four dimensions and the customer-audit table in section (a) are a usable starting point
5 · Leadership	Engineering leadership is engaged and technically credible, but there are **no quality KPIs and no leadership-owned quality review**	A named leadership owner for a recurring quality review
6 · Planning / risk-based thinking	Key risks (config-server single point of failure, secret exposure, key-person dependency) are known individually but **not managed systematically**	A risk register with prioritized mitigations. This roadmap is effectively its first version
7.1.2 · People **7.1.6 · Organizational knowledge**	This is where the **key-person dependency** appears in 9001 terms. The clause requires that the organization determine and provide the persons necessary for effective operation, and that it determine and **maintain the knowledge necessary for its processes**. Today the commercial platform's operating knowledge is **largely undocumented and held by one engineer** (no runbooks)	Runbooks and cross-training are a 9001 requirement, not only good practice. The staffing recommendation in section (d) sits here
7.2 · Competence	Deep and demonstrable competence on the scientific side. Software-engineering competence is **thin on the platform side by headcount rather than by capability**	Define required competences per role, which also gives the hiring plan an auditable basis
7.5 · Documented information	No Definition of Ready or Done, no defined shared process, documentation scattered across Notion and repositories, no runbooks	Define and document the minimum process (DoR/DoD, PR template, runbooks)
8 · Operational control	Release and deploy proceed without effective gates: CI runs, and on the cloud it does not block	Mandatory gates on protected branches, starting with the one-line Sonar change
9 · Performance evaluation	No structured monitoring or measurement, and no management review cycle. **Partly addressed since this review:** the software-scope internal-audit self-assessment has now been drafted and is under internal review, which puts the first 9.2 record in progress. What remains is the measurement layer (9.1) and a recurring management review (9.3)	A KPI dashboard plus a periodic management review, closing the 9.1 and 9.3 loop
10 · Improvement / corrective action	Incidents are handled manually and case by case, with no structured corrective-action process and no incident record	A continuous-improvement loop (PDCA); an incident log makes corrective action possible and also unlocks two DORA metrics

Where to focus: in 9001 terms the opportunity concentrates in **organizational knowledge (7.1.6), documented information (7.5), operational control (8), performance evaluation (9) and improvement (10)**. Worth highlighting for anyone reading this from a quality-management background: **the staffing and runbook recommendations in this report are not a stylistic preference, they map directly onto clause 7.1**. In PDCA language, Corintis has a strong "Do" and now gets to formalize "Plan, Check and Act" around it. That is a favorable starting position, because the hard part, an engineering team that reliably delivers, is already in place. This diagnostic is, in effect, the first "Check", and the drafted software-scope self-assessment is the first internal-audit record taking shape alongside it.

SECTION C

Prioritized quality roadmap · P1 to P3

Guiding principle: **sequence in waves, never open four fronts at once** (team capacity is the constraint). Order: **people → process → metrics**. Effort: S (days) · M (weeks) · L (a month or more).

> **Constraint.** A company-wide hiring freeze is in effect (Product Owner and DevSecOps on hold). The root cause does not change, it becomes more urgent, but the order adapts: **de-risk first with current staff** (cross-training, runbooks, low-cost process fixes), with the hiring business case ready for the moment the freeze lifts.

P1 · Immediate (containment and unlocking)

#	Action	Effort	Dim.	Depends on
1	Rotate the Zendesk token, purge git history, enable org-wide **secret scanning and push protection**	S	D1	Org admin
1b	Enable `sonar.qualitygate.wait=true` on the frontend and backend so the already-strict quality gate can actually fail CI, then add that check to branch protection as a required status check. **One line of configuration: no new tooling, no headcount, unaffected by the freeze**	S	D1	-
2	Protect the unprotected repositories (especially `prod.properties` and `config-server`) and fix config-server CI (boot the Spring context in CI; remove the blind `-DskipTests`)	S-M	D1	-
3	**De-risk the bus factor without hiring**: cross-train the full-stack engineer onto the backend for redundancy, plus cloud runbooks/documentation (get the platform out of one person's head)	M	D4	-
3b	Prepare the **hiring business case** (cloud/backend engineer + DevSecOps) as priority #1 for when the freeze lifts: business continuity, enables cold-plate performance	S	D4	Leadership
3c	With the interim Product Owner, establish **one intake and one prioritized backlog** for product demand, replacing person-to-person routing. Two design rules matter more than the tool choice: keep *discovery* (customer opportunities, not yet committed) visibly separate from *delivery* (committed work), and **do not introduce a fourth tool**. The organization already runs GitHub Projects and Notion; adding a new platform would worsen the fragmentation this dimension identifies. Revisit tooling once the role is full-time	S-M	D2	Interim PO

P2 · Foundational (sequenced by capacity)

#	Action	Effort	Dim.	Depends on
4	**DevSecOps**: on hold during the freeze; in the interim, assign its responsibilities (secret scanning, gates, environment separation, config server) as an explicit shared interim duty; hire as soon as the freeze lifts	L	D4→D1	Freeze
5	**Tests as a required status check** on protected branches (start: solver `develop`, frontend, backend); mandatory review once a second cloud engineer exists	M	D1/D2	#4, headcount
6	Structure the ad hoc validation into a **V&V 20 program** (baseline vs ANSYS/COMSOL/Fidelity plus uncertainty quantification, documented) and make verification a release gate	L	D1	-
7	Test baseline for money/compute-critical services (`monitor.credits`, `job.scheduler`)	M	D1	#4
8	**Minimum KPI dashboard** (review rate, share of gated repos, lead time, coverage) with a leadership owner	M	D3	-

P3 · Maturing

#	Action	Effort	Dim.
9	Cloud release discipline (semver, tags, changelog); instrument deploy and incident events → **full DORA**	M-L	D2/D3
10	Decouple the mathematics from the solver stack (verify forms early rather than late as silently wrong physics); publish a **V&V plan**; add a staging tier (dev/prod already separated) and separate the solver's ad hoc compute (GCP VMs)	M	D1
11	*(Later phase / advisory)* IP: Firedrake LGPL license (legal track in motion), artifact signing/SBOM, desktop binary hardening	L	D1

Critical cross-cutting dependency: the review/gate items (#5) depend on **adding people** (#3, #4). That is why the roadmap puts staffing first.

Where this leads

Completing P1 and P2 puts Corintis in a materially different commercial position, and it is worth being concrete about what that looks like. The engineering organization would be able to state, with evidence rather than assurance, that:

- simulation results carry a **documented validation basis and a quantified uncertainty envelope**, referenced to ASME V&V 20, which is the language a semiconductor or defense customer's own engineers will use to interrogate the tool;
- no change reaches production without passing a **defined, enforced gate**, and that gate can be demonstrated in an audit rather than described;
- the commercial platform is **not dependent on any single individual**, which is the answer procurement is looking for when it asks about continuity of supply;
- delivery and quality performance are **measured and reviewed by leadership** on a defined cadence, closing the ISO 9001 management-review loop.

That is the difference between a company with excellent technology and a company that can be **qualified as a strategic supplier**. The technology is already there. This roadmap is about making the engineering organization legible to buyers of that size, and it is deliberately sequenced so that the team can absorb it without slowing the cold-plate performance work that the business is actually judged on.

SECTION D

Roles recommendation & RACI

Roles to bring in: Cloud software engineer(s) *(the #1 structural fix)* · DevSecOps · Product Owner *(approved, on hold during the freeze)* · QA/QE *(AI-tooling-leveraged)*. **In place:** Head of Software Engineering, Director of Global Quality (as partner).

Team Topologies (target): keep the **solver stream-aligned** (it works); build a **stream-aligned cloud/product team** with real headcount; a thin **platform capability** (CI/CD, config, deploy, environments) where DevSecOps lives; and an **enabling** quality function (QA that coaches rather than gatekeeps).

Activity	R	A	C	I
Backlog / refinement / DoR-DoD	PO	Head of SW Eng	Cloud SWE, CompEng	Leadership
Peer code review (enforced)	Cloud SWE, CompEng	Head of SW Eng	QA	-
Test strategy & coverage	QA, engineers	Dir. Global Quality	Head of SW Eng	Leadership
Solver V&V program (V&V 20)	CompEng	Head of SW Eng	Dir. Global Quality, QA	Leadership
CI/CD & release gates	DevSecOps	Head of SW Eng	QA	Cloud SWE
Deploy & environment separation	DevSecOps	Head of SW Eng	Cloud SWE	-
Secrets / security / IP	DevSecOps	Head of SW Eng	Dir. Global Quality	Leadership
Quality KPI framework & dashboard	QA	**Dir. Global Quality**	Head of SW Eng, PO	Leadership
Config management	DevSecOps	Head of SW Eng	Cloud SWE	-

R = Responsible · A = Accountable · C = Consulted · I = Informed. CompEng = solver team engineers; Cloud SWE = cloud engineers including new hires.

SECTION E

Annex · Evidence reviewed

Method. Read-only analysis of 16 cloned repositories: a structured first-pass sweep (10-field checklist per repository); deep dives on solver V&V and on release gates; flow metrics via the GitHub GraphQL API; authorship and release signals via local git; branch protection via the client's settings report (2026-07-03); a structured interview with the Head of Software Engineering (2026-07-06) plus the platform operations review (corrected version, 2026-08), written platform-owner answers on deployment (2026-07-06), and the **SonarCloud technical-health report** (generated 2026-08-04; underlying analysis 2026-07-31, revision `d127aac`) cross-checked against the CI workflow definitions.

Repositories in scope (16). Solver core: `glaciercore`, `firedrake-wheels`. Desktop: `glacierware`. Cloud: `app.frontend.corintis`, `app.backend.corintis`, `config-server.cloud`, `glaciercore.json.builder`, `glaciercore.output.formatter`, `glacierware-prod.properties`, `glacierware.email.notification`, `glacierware.fast-api`, `glacierware.job.scheduler`, `glacierware.monitor.{credits,job,sweep}`, `glacierware.task.worker`.

Evidence per finding (examples). V&V: `glaciercore/tests/verification/test_physics.py` (MMS, convergence order >1.9). Gates: `react.js.yml` (deploy `needs: sonarcloud`) vs `config-server.cloud/.github/workflows/ci.yml` (`-DskipTests`); branch-protection report (0/16 with required tests). Secret: `glacierware-prod.properties/application-{dev,prod}.properties`, line 17. Bus factor: `git shortlog` (a single author across ~12 of 16 repositories; 27 distinct authors on `glaciercore`). Flow: GraphQL (review rate solver 80% vs backend 0%).

Data still pending (does not block the diagnostic). Backend Sonar metrics and coverage (the same health report, re-run with the `Corintis_app.backend.corintis` project scope, would close this); GitHub Projects usage (access token lacked the `read:project` scope). Closed on 2026-08-05: the exact SonarCloud thresholds are now documented, and frontend coverage is established at 86.1%. Resolved during the engagement: Heroku "Wait for CI" is ON and dev/prod are separate apps with separate databases; the operations review was validated in its corrected 2026-08 version, closing both open discrepancies (`monitor.opt` does not exist as a service, optimization runs inside `monitor.job`; `task.worker` is in scope and current).

Acceptance criteria (met). The four dimensions covered at the agreed depth; every finding traced to evidence; the roadmap prioritized and estimated; and the software-scope ISO 9001 internal-audit self-assessment drafted. Pending: the review session with the quality organization, and the results presentation sessions to leadership and to the engineering team.